Informe técnico: Trendy API (Backend NestJS)

Universidad Icesi · Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas Departamento de Computación y Sistemas Inteligentes · Computación en Internet III

Taller: Backend – NestJS

Docente: Gustavo Adolfo Gonzalez Medina

Integrantes:

- Camilo Andrés Martínez Moreno - A00405205
- Martín Borrero Herrera - A00403871
- Javier Andrés Paz Luna - A00405619

Repositorio: https://github.com/andresMC21/Trendy

Aplicación desplegada: https://trendy-app-web-un9c.onrender.com/api

Documentación Swagger: https://trendy-app-web-un9c.onrender.com/api/docs

1. Introducción

Trendy es un marketplace hiperlocal de segunda mano: las personas publican productos, los encuentran por barrio o por cercanía geográfica, acuerdan la entrega en un punto seguro y se califican entre sí cuando la transacción termina. Este documento describe el backend que sirve esa aplicación: una API REST construida con NestJS, que persiste sus datos en PostgreSQL con PostGIS mediante TypeORM, y que incluye pruebas unitarias y de integración, documentación Swagger, pipelines de integración continua y despliegue en la nube.

El informe describe cada endpoint (parámetros y respuestas) y explica cómo se implementaron la autenticación, la autorización y la persistencia en la base de datos, así como la ejecución de las pruebas.

1.1 Cumplimiento de los requisitos del taller

Requisito	Estado	Cómo se cumple
Seed con usuario admin	Cumple	Script `npm run seed` (`src/seed`): crea admin, moderador y dos usuarios, 3 productos, 1 transacción y 1 reseña. Se puede repetir sin duplicar datos
Autenticación con JWT	Cumple	Registro e inicio de sesión que devuelven un JWT de 1 hora; rutas protegidas con el decorador `@Auth()` (sección 3)

Requisito	Estado	Cómo se cumple
Autenticación con 2FA	No implementado	Queda como trabajo futuro (sección 11)
Cerrar sesión	Parcial	La sesión es sin estado: el cliente descarta el token. No existe un endpoint de logout ni lista de revocación (sección 3.5)
Rutas protegidas	Cumple	27 de los 33 endpoints exigen JWT
Autorización por roles	Cumple	Roles `ADMIN`, `MODERATOR` y `USER`; asignación mediante `PATCH /api/admin/users/:id/role` (sección 4)
Pruebas con cobertura mínima del 80 %	Cumple	168 pruebas unitarias y 38 de integración; cobertura 91.75 % sentencias, 80.98 % ramas, 86.25 % funciones y 91.37 % líneas (sección 8)
Persistencia con ORM	Cumple	TypeORM sobre PostgreSQL/PostGIS, con migraciones (sección 5)
Funcionalidades y colección Postman	Cumple	33 endpoints; colección con 179 requests y 260 aserciones
Swagger en los endpoints	Cumple	`/api/docs`
GitHub Actions para ejecutar pruebas	Cumple	Workflows `tests` y `PROD-CI` en cada push y pull request (sección 9)
Despliegue en la nube con pipelines	Cumple	Render, desplegado automáticamente desde GitHub Actions (sección 9)

2. Arquitectura y tecnologías

2.1 Tecnologías

Capa	Tecnología
Framework	NestJS 12 (TypeScript, Node.js 24)

Capa	Tecnología
Persistencia	PostgreSQL 16 + PostGIS 3.4, TypeORM, migraciones
Autenticación	Passport (`passport-jwt`), `@nestjs/jwt`, bcrypt
Validación	`class-validator` y `class-transformer` (ValidationPipe global)
Documentación	Swagger / OpenAPI (`@nestjs/swagger`)
Pruebas	Jest (unitarias), Supertest (integración), Postman/Newman (manuales y regresión)
CI/CD y despliegue	GitHub Actions, Render

2.2 Módulos

La aplicación se divide en módulos de NestJS, cada uno con sus controladores, servicios, DTO y entidades:

Módulo	Responsabilidad	Rutas base	Depende de
`UserModule`	Registro, autenticación, perfil, administración de usuarios y moderación de verificaciones	`/user`, `/admin/users`, `/moderation/verifications`	— (exporta Passport, JWT y `UserService`)
`ProductModule`	Productos, feed geográfico y favoritos	`/products`	`UserModule` (exporta `ProductService`)
`TransactionModule`	Compra o trueque y máquina de estados	`/transactions`	`UserModule`, `ProductModule`
`ReviewModule`	Reseñas y promedio de calificación	`/review`	`UserModule`
`AppModule`	Configuración global, conexión a la base de datos y `/health`	`/health`	todos los anteriores

La dependencia más importante es la de transacciones con productos: cuando una transacción cambia de estado, `TransactionService` invoca `ProductService.updateStatus` para reservar, vender o liberar el producto.

2.3 Ciclo de una petición

1. **Prefijo global `/api`** y CORS habilitado.

2. **ValidationPipe global** (`whitelist`, `forbidNonWhitelisted`, `transform`): rechaza con 400 los campos desconocidos y los valores inválidos, y convierte tipos (por ejemplo, `limit=5` en número).
3. **Autenticación**: si la ruta usa `@Auth()`, `AuthGuard('jwt')` extrae el token Bearer y `JwtStrategy` carga al usuario desde la base de datos.
4. **Autorización**: `UserRoleGuard` compara los roles del usuario con los roles exigidos por la ruta.
5. **ParseUUIDPipe** valida los parámetros `:id`.
6. **Controlador → servicio → TypeORM → PostgreSQL.**
7. Cualquier excepción se serializa con un cuerpo común: `{ "statusCode": 409, "message": "...", "error": "Conflict" }` (en errores de validación, `message` es una lista de textos).

2.4 Configuración

Todo se configura por variables de entorno (`ConfigModule` global):

Variable	Uso
`DATABASE_URL`	Cadena de conexión a PostgreSQL (alternativa: `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USERNAME`, `DB_PASSWORD`)
`JWT_SECRET`	Secreto de firma de los JWT. Es obligatorio: sin él la aplicación no arranca
`PORT`	Puerto HTTP (3000 por defecto)
`DB_SSL`	Fuerza o desactiva SSL; en `NODE_ENV=production` el SSL se activa por defecto
`DB_SYNCHRONIZE` / `TYPEORM_SYNCHRONIZE`	Sincronización automática del esquema (desactivada por defecto: se usan migraciones)
`FEED_CACHE_TTL_SECONDS`	Duración de la caché del feed de productos (30 s por defecto; 0 la desactiva)

3. Autenticación

La autenticación se basa en **JSON Web Tokens** emitidos por la propia API y verificados con Passport.

3.1 Registro e inicio de sesión

- `POST /api/user/register` normaliza el correo (minúsculas y sin espacios), valida los datos, cifra la contraseña y crea una cuenta con rol `USER`, activa y con verificación `UNVERIFIED`. Responde con el perfil y un JWT, de modo que el usuario queda autenticado al registrarse. Si el correo ya existe responde 409.
- `POST /api/user/login` busca al usuario por correo, compara la contraseña con el hash y exige que la cuenta esté activa. Responde `{ id, email, token }`. Cuando el correo no existe, la contraseña es incorrecta o la cuenta está inactiva, devuelve el **mismo** error 401 (`Email or password incorrect`) para no revelar cuál de las condiciones falló.

Ejemplo de respuesta del registro (el token se abrevia):

```
{
  "id": "1fe103a7-30bb-4777-95dd-fec37e4c6c85",
  "email": "nuevo@example.com",
  "fullName": "Nuevo Usuario",
  "photoUrl": null,
  "info": null,
  "role": ["USER"],
  "verificationStatus": "UNVERIFIED",
  "isActive": true,
  "ratingAverage": 0,
  "reviewsCount": 0,
  "createdAt": "2026-10-12T07:17:20.778Z",
  "updatedAt": "2026-10-12T07:17:20.778Z",
  "token": "eyJhbGciOiJIUzI1NiIsInR5c..."
}
```

3.2 El token JWT

- Se firma con el algoritmo HS256 usando `JWT_SECRET` y **expira a la hora** de emitido (`expiresIn: '1h'`).
- El *payload* contiene el identificador y el correo del usuario, además de las marcas estándar `iat` y `exp`:

```
{ "id": "44444444-4444-4444-8444-444444444444", "email": "carlos@trendy.local",
"iat": 1791789441, "exp": 1791793041 }
```

- El cliente lo envía en cada petición protegida: `Authorization: Bearer <token>`.

3.3 Rutas protegidas

Las rutas que requieren autenticación se marcan con el decorador compuesto `@Auth()` (sección 4.2). Su parte de autenticación usa `AuthGuard()` de Passport con la estrategia `JwtStrategy`, que en cada petición:

1. Extrae el token del encabezado `Authorization` (esquema Bearer) y verifica firma y expiración. Si falta o es inválido, responde **401.**
2. Vuelve a consultar al usuario en la base de datos con el `id` del token. Si ya no existe, responde 401 (`Token not valid`).
3. Si el usuario está inactivo, responde 401 (`User is not active`).
4. Deja el usuario completo en `request.user`, disponible para los controladores mediante el decorador `@GetUser()`.

Como el usuario se consulta en cada petición, los cambios administrativos tienen efecto inmediato: una cuenta desactivada pierde el acceso con los tokens que ya tenía emitidos, y un cambio de rol se aplica sin necesidad de volver a iniciar sesión.

3.4 Manejo de contraseñas

- Se almacenan cifradas con bcrypt (10 rondas); nunca en texto plano.

- La columna `password` está marcada `select: false`: TypeORM no la incluye en ninguna consulta, salvo en las dos operaciones que la necesitan (login y verificación de la contraseña actual).
- Las respuestas de la API usan DTO con `@Expose()` explícito (`UserProfileDto`), por lo que nunca incluyen la contraseña.
- Política: entre 8 y 12 caracteres. Cambiar la contraseña o el correo exige conocer la contraseña actual (si no coincide, 403), y la nueva contraseña debe ser distinta de la actual.

3.5 Cierre de sesión y 2FA (alcance)

La sesión es sin estado (*stateless*): el servidor no guarda tokens, por lo que cerrar sesión consiste en que el cliente descarte el token. El acceso queda limitado por la expiración de 1 hora y, ante un incidente, por la desactivación de la cuenta (`PATCH /api/admin/users/:id/active`), que invalida de inmediato todos sus tokens. No se implementaron un endpoint de logout con lista de revocación, *refresh tokens* ni autenticación de dos factores (2FA); se contemplan como trabajo futuro (sección 11).

3.6 Verificación de identidad

Además de la autenticación, cada usuario tiene un estado de verificación de identidad: `UNVERIFIED` → `PENDING` (la persona la solicita con `POST /api/user/me/verification`) → `VERIFIED` o `REJECTED` (la resuelve un moderador o administrador en `PATCH /api/moderation/verifications/:userId`). Una persona rechazada puede volver a solicitarla. Mientras la verificación está `PENDING` o `VERIFIED`, el nombre del perfil no puede cambiarse.

4. Autorización

4.1 Roles

Existen tres roles, definidos en el enum `ValidRoles` y almacenados como un arreglo en la columna `user.role` (un usuario puede tener varios; el valor por defecto es `USER`):

Rol	Descripción
`USER`	Persona usuaria de la aplicación: publica, compra, reseña
`MODERATOR`	Revisa solicitudes de verificación, consulta usuarios (solo lectura) y puede eliminar productos ajenos
`ADMIN`	Administra usuarios: edita datos, asigna roles, activa o desactiva cuentas; además tiene las capacidades del moderador

4.2 Mecanismo

La autorización se implementa con tres piezas que se aplican juntas mediante el decorador `@Auth(...roles)`:

- **`@RoleProtected(...roles)`**: guarda en los metadatos de la ruta los roles permitidos.
- **`AuthGuard()`**: exige un JWT válido (sección 3.3).
- **`UserRoleGuard`**: lee con `Reflector` los roles requeridos y los compara con `request.user.role`. Si no hay ninguno en común, responde **403** (`No tiene permisos para este recurso`); si la ruta no exige roles, solo se pide estar autenticado.

```
@Get()
@Auth(ValidRoles.ADMIN, ValidRoles.MODERATOR)   // JWT válido + rol ADMIN o MODERATOR
findAll(@Query() query: AdminUserQueryDto) { ... }
```

Cada operación en Swagger indica qué roles necesita y qué códigos de error puede devolver.

4.3 Asignación de roles (mecanismo de administración)

Los roles **no** se pueden elegir al registrarse: el registro siempre crea un `USER` y el ValidationPipe rechaza cualquier intento de enviar un campo `role`. Los roles se asignan únicamente mediante el endpoint de administración `PATCH /api/admin/users/:id/role`, que solo puede usar un `ADMIN` y que reemplaza la lista completa de roles del usuario. El primer administrador lo crea el *seed* (`admin@trendy.local`).

Protecciones adicionales: un administrador no puede quitarse a sí mismo el rol `ADMIN`, ni desactivar o eliminar su propia cuenta, para evitar quedarse sin administradores.

4.4 Matriz de permisos

Recurso / acción	Público	Autenticado	MODERATOR	ADMIN
Registro, login, health	✔	✔	✔	✔
Ver feed y detalle de productos	✔	✔	✔	✔
Perfil propio, cambio de contraseña y correo, solicitar verificación, ver perfil de otro usuario		✔	✔	✔
Publicar producto, marcar favoritos, iniciar compra, escribir reseñas		✔	✔	✔
Editar un producto		solo el dueño	solo si es el dueño	solo si es el dueño
Eliminar un producto		solo el dueño	✔ (cualquiera)	✔ (cualquiera)

Recurso / acción	Público	Autenticado	MODERATOR	ADMIN
Ver una transacción o actualizar su estado		solo comprador y vendedor	solo si participa	solo si participa
Editar o eliminar una reseña		solo su autor	solo si es el autor	solo si es el autor
Listar y consultar usuarios (`/admin/users`)			✔	✔
Listar y resolver verificaciones			✔ (no la propia)	✔ (no la propia)
Editar usuarios, asignar roles, activar/desactivar, eliminar				✔

4.5 Reglas de pertenencia

Además de los roles, los servicios aplican reglas sobre quién es dueño o participante del recurso: solo el vendedor edita su producto; solo el comprador y el vendedor ven o modifican una transacción; solo el autor edita o borra su reseña, y solo el autor y la persona reseñada pueden verla; un moderador no puede resolver su propia solicitud de verificación; el vendedor no puede comprar su propio producto.

5. Persistencia en la base de datos

5.1 Tecnología y conexión

Se usa TypeORM con el driver `pg` sobre PostgreSQL 16 con la extensión PostGIS. La conexión se configura en `AppModule` con `TypeOrmModule.forRootAsync`, a partir de `DATABASE_URL` (o de las variables `DB_*`), con SSL opcional y `autoLoadEntities` para registrar las entidades de cada módulo. Los servicios acceden a los datos con repositorios (`@InjectRepository`) y con `QueryBuilder` para las consultas más elaboradas (feed geográfico, conteo de favoritos, promedio de calificaciones).

5.2 Modelo de datos

Hay cinco tablas de negocio más la tabla `migrations` de TypeORM. Los identificadores son UUID generados por la base de datos.

`user`

Columna	Tipo	Notas
`id`	uuid	PK

Columna	Tipo	Notas
`email`	text	Único; se normaliza a minúsculas antes de guardar
`password`	text	Hash bcrypt; `select: false`
`fullName`, `photoUrl`, `info`	text	`photoUrl` e `info` opcionales
`role`	enum[]	`ADMIN`, `USER`, `MODERATOR`; por defecto {`USER`}
`verificationStatus`	text	`UNVERIFIED` por defecto
`ratingAverage`, `reviewsCount`	float, integer	Se recalculan al crear, editar o borrar reseñas
`isActive`	boolean	`true` por defecto; base del borrado lógico
`createdAt`, `updatedAt`	timestamp	Automáticos

`product`

Columna	Tipo	Notas
`id`	uuid	PK
`sellerId`	uuid	FK → `user`; indexada
`title`, `description`, `category`	text	`category` indexada
`price`	numeric(12,2)	0 si es donación o trueque
`acceptsBarter`	boolean	
`condition`	text	`NUEVO`, `COMO_NUEVO`, `USADO`
`status`	text	`AVAILABLE`, `RESERVED`, `SOLD`, `REMOVED`; indexada
`location`	geography(Point, 4326)	Columna PostGIS con índice espacial GiST
`neighborhoodId`	text	Filtro hiperlocal; indexada
`estimatedWeightKg`	numeric(8,2)	Opcional (base para métricas de impacto)
`viewsCount`	integer	Se incrementa en cada consulta del detalle

Columna	Tipo	Notas
`createdAt`, `updatedAt`	timestamp	Automáticos

`product_favorites` (relación N:M entre `product` y `user`): `productId` + `userId` como llave primaria compuesta.

`transaction`

Columna	Tipo	Notas
`id`	uuid	PK
`productId`, `buyerId`, `sellerId`	uuid	FK → `product` y `user` (dos veces); las tres indexadas
`status`	text	`PENDING`, `AGREED_POINT`, `COMPLETED`, `CANCELLED`; indexada
`safePointId`	text	Se llena al confirmar el punto seguro
`scheduledTime`	timestamptz	Horario acordado de encuentro
`createdAt`, `updatedAt`	timestamp	Automáticos

`review`

Columna	Tipo	Notas
`id`	uuid	PK
`transactionId`	uuid	Transacción reseñada
`reviewerId`, `targetUserId`	uuid	FK → `user` (autor y persona reseñada)
`rating`	integer	1 a 5 (validado en la API)
`comment`	text	Opcional
`createdAt`	timestamp	Automático
		Restricción única (`transactionId`, `reviewerId`): una reseña por participante y transacción

5.3 Relaciones

```
user 1 ──────── N product            (vendedor)
user N ──────── N product            (favoritos, tabla product_favorites)
```

```
product 1 ──── N transaction
user 1 ──────── N transaction        (como comprador)
user 1 ──────── N transaction        (como vendedor)
transaction 1 ──── N review         (a lo sumo una por participante)
user 1 ──────── N review             (como autor)
user 1 ──────── N review             (como persona reseñada)
```

5.4 Datos geográficos con PostGIS

La ubicación del producto se guarda como `geography(Point, 4326)` (coordenadas WGS 84), que permite medir distancias reales en metros. TypeORM la lee y escribe como GeoJSON (`[longitud, latitud]`), y la API la expone como `{ lat, lng }`. El feed usa dos funciones de PostGIS:

- `ST_DWithin(location, punto, metros)` filtra los productos dentro del radio pedido (`radius`, en km, convertido a metros); el índice GiST lo acelera.
- `ST_Distance(location, punto)` calcula la distancia a cada producto, que se devuelve como `distanceKm` y permite ordenar con `sort=distance`.

5.5 Migraciones y esquema

El esquema se versiona con migraciones de TypeORM (`src/database/migrations`), no con sincronización automática (`synchronize` está desactivado por defecto). La migración inicial crea el tipo enum de roles, las cinco tablas, los índices (incluido el espacial) y las llaves foráneas. Comandos:

Comando	Uso
`npm run migration:run`	Aplica las migraciones pendientes (desarrollo y CI)
`npm run migration:generate -- src/database/migrations/Nombre`	Genera una migración a partir de los cambios en las entidades
`npm run migration:run:prod`	Aplica las migraciones desde el código compilado (`dist`); lo ejecuta Render antes de iniciar la API

Al conectarse, TypeORM instala las extensiones `postgis` y `uuid-ossp` si no existen.

5.6 Borrado lógico e integridad

Nada importante se borra físicamente: eliminar un usuario (`DELETE /api/admin/users/:id`) lo desactiva (`isActive = false`) y eliminar un producto cambia su estado a `REMOVED`. Así se conserva el historial de transacciones y reseñas. Las llaves foráneas garantizan la integridad referencial entre tablas.

5.7 Datos iniciales (seed)

`npm run seed` ejecuta `src/seed/seed.ts`, que dentro de una transacción de base de datos inserta o actualiza (con identificadores fijos, por lo que se puede repetir) los siguientes datos:

Correo	Contraseña	Rol
`admin@trendy.local`	`Admin1234`	ADMIN
`moderador@trendy.local`	`Mod12345`	MODERATOR

Correo	Contraseña	Rol
`ana@trendy.local`	`Ana12345`	USER (verificada; vende 2 productos)
`carlos@trendy.local`	`Carlos12`	USER (vende 1 producto y compró uno de Ana)

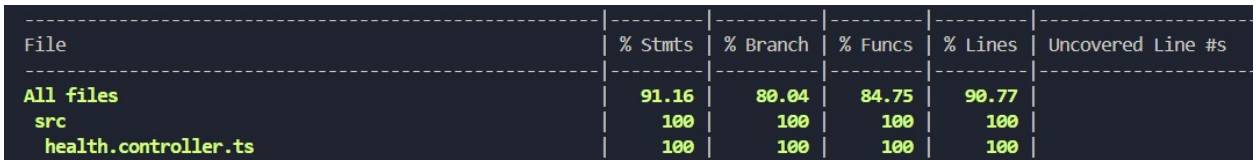
Además, 3 productos, 1 transacción completada y 1 reseña, que sirven para probar la API sin preparar datos a mano.

5.8 Caché del feed

Para reducir consultas repetidas, `GET /api/products` guarda sus resultados en una caché en memoria con vigencia de 30 segundos (configurable). La caché se invalida cuando un producto del barrio se crea, edita, elimina o cambia de estado.

5.9: coverage

Al usar el comando npm run test:cov, obtenemos lo siguiente



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	91.16	80.04	84.75	90.77	
src	100	100	100	100	
health.controller.ts	100	100	100	100	

Esto dandonos mas del 80% que nos exige el taller(este comprueba las pruebas unitarias)